Python Engineering - Class 07 Reference Guide

NumPy Arrays

Reference guide to the syntax and tools used in this class. It explains what each element does, how it is written and when it is useful.

Quick map

Concept	Syntax	Idea
np.array()	np.array(data)	Create a NumPy array from compatible data.
shape	A.shape	Tuple describing length along every structural axis.
ndim	A.ndim	Number of array axes, not the physical dimension of the vector.
size	A.size	Total number of elements.
dtype	A.dtype	Element data type used by the array.
indexing	A[i,j] / A[:,j]	Select elements, rows or columns.
reshape()	A.reshape(r,c)	View/reorganize data with a compatible shape.
transpose	A.T	Swap axes for a 2D matrix.
np.vstack()	np.vstack((A,B))	Stack arrays vertically.
np.hstack()	np.hstack((A,B))	Stack arrays horizontally.
broadcasting	A + vector	Automatic compatible shape expansion.
* vs @	A*B / A@B	Element-wise multiplication vs matrix product.
axis	np.max(A,axis=0)	Select which structural axis is reduced.

1. np.array()

Syntax: `np.array(data)`

Create a NumPy array from compatible data.

```
trajectory = np.array(rows)
```

Control question: before running the example, predict the result type and shape.

2. shape

Syntax: `A.shape`

Tuple describing length along every structural axis.

```
print(trajectory.shape)
```

Control question: before running the example, predict the result type and shape.

3. ndim

Syntax: `A.ndim`

Number of array axes, not the physical dimension of the vector.

```
print(trajectory.ndim)
```

Control question: before running the example, predict the result type and shape.

4. size

Syntax: `A.size`

Total number of elements.

```
print(trajectory.size)
```

Control question: before running the example, predict the result type and shape.

5. dtype

Syntax: `A.dtype`

Element data type used by the array.

```
print(trajectory.dtype)
```

Control question: before running the example, predict the result type and shape.

6. indexing

Syntax: `A[i,j] / A[:,j]`

Select elements, rows or columns.

```
heights = trajectory[:, 0]
```

Control question: before running the example, predict the result type and shape.

7. reshape()

Syntax: `A.reshape(r,c)`

View/reorganize data with a compatible shape.

```
row = vector.reshape(1, 3)
```

Control question: before running the example, predict the result type and shape.

8. transpose

Syntax: `A.T`

Swap axes for a 2D matrix.

```
AT = A.T
```

Control question: before running the example, predict the result type and shape.

9. np.vstack()

Syntax: `np.vstack((A,B))`

Stack arrays vertically.

```
A = np.vstack((A, new_row))
```

Control question: before running the example, predict the result type and shape.

10. np.hstack()

Syntax: `np.hstack((A,B))`

Stack arrays horizontally.

```
AB = np.hstack((A, B))
```

Control question: before running the example, predict the result type and shape.

11. broadcasting

Syntax: `A + vector`

Automatic compatible shape expansion.

```
shifted = A + np.array([1.0, 0.0, 0.0])
```

Control question: before running the example, predict the result type and shape.

12. * vs @

Syntax: `A*B / A@B`

Element-wise multiplication vs matrix product.

```
elementwise = A * B
matrix_product = A @ B
```

Control question: before running the example, predict the result type and shape.

13. axis

Syntax: `np.max(A,axis=0)`

Select which structural axis is reduced.

```
column_max = np.max(A, axis=0)
```

Control question: before running the example, predict the result type and shape.

Common mistakes

- Confusing a function name with a function call using parentheses.
- Not checking types, indices or shapes before an operation.
- Using a tool because it works without being able to explain what it receives and returns.

Mental cheat sheet

```
np.array(): np.array(data)
shape: A.shape
ndim: A.ndim
size: A.size
```

```
dtype: A.dtype
indexing: A[i,j] / A[:,j]
reshape(): A.reshape(r,c)
transpose: A.T
np.vstack(): np.vstack((A,B))
np.hstack(): np.hstack((A,B))
broadcasting: A + vector
* vs @: A*B / A@B
axis: np.max(A,axis=0)
```